

C# 프로그래밍

동명대학교 게임학부 · 2026년 1학기

Lecture 4

C# 클래스(Class) 이해하기

Console 프로그래밍에서 Life Game까지

강영민

동명대학교 게임학부

Contents

1 강의 개요	2
1.1 학습 목표	2
1.2 강의 구성	2
1.3 왜 클래스인가?	2
2 VS Code 환경 설정	2
2.1 필요 도구 설치	2
2.2 Console 프로젝트 생성	3
2.3 빌드 및 실행	3
3 클래스 기초 — 첫 번째 클래스 만들기	3
3.1 클래스란?	3
3.2 Step 1: 가장 간단한 클래스	4
3.3 Step 2: 메서드 추가하기	4
4 생성자와 프로퍼티	5
4.1 생성자 (Constructor)	5
4.2 프로퍼티 (Property)	6
5 클래스 활용 — 객체 배열과 상호작용	8
5.1 객체 배열	8
5.2 클래스 안에서 다른 클래스 사용하기	10
6 Console 시각화 — 텍스트로 그림 그리기	12
6.1 Console 색상 출력	12
6.2 커서 위치 제어	13
6.3 Canvas 클래스 — 화면을 객체로 관리하기	13
6.4 Canvas 클래스에 도형 메서드 추가하기	15
7 종합 프로젝트 — Conway's Game of Life	17
7.1 Game of Life란?	17
7.1.1 규칙 (매우 간단!)	17
7.2 Step 1: Cell 클래스	17
7.3 Step 2: Grid 클래스 — 격자 관리	17
7.4 Step 3: 초기 패턴 설정	20
7.5 Step 4: 실행 — 모든 것을 합치기	21
8 핵심 개념 정리	24

1. 강의 개요

1.1. 학습 목표

이번 강의에서는 C#의 핵심 개념인 **클래스(Class)**를 단계적으로 학습한다. Console 기반의 간단한 예제에서 출발하여, 최종적으로 **Conway's Game of Life**를 구현하면서 클래스의 실용적 활용을 체험한다.

- 클래스와 객체의 개념을 이해하고 직접 정의할 수 있다.
- 필드, 메서드, 생성자, 프로퍼티의 역할을 구분할 수 있다.
- 여러 객체를 배열로 관리하고, 객체 간 상호작용을 구현할 수 있다.
- Console 화면에 시각적 출력을 만들어 클래스 설계를 확인할 수 있다.
- Life Game을 클래스 기반으로 설계하고 구현할 수 있다.

1.2. 강의 구성

파트	내용
1부	VS Code 환경 설정 및 Console 프로젝트 생성
2부	클래스 기초 — 필드, 메서드, 객체 생성
3부	생성자와 프로퍼티
4부	클래스 활용 — 배열, 객체 간 상호작용
5부	Console 시각화 — 색상과 커서를 활용한 그래픽 출력
6부	종합 프로젝트 — Conway's Game of Life

1.3. 왜 클래스인가?

그래픽 디자인에서 레이어(Layer)를 떠올려 보자. 하나의 레이어에는 위치, 크기, 불투명도, 블렌드 모드 같은 **속성(데이터)**이 있고, 이동, 회전, 크기 조절 같은 **동작(기능)**이 있다.

클래스는 바로 이런 “데이터 + 기능”을 하나로 묶는 도구이다.

디자인 세계	C# 클래스	설명
레이어 템플릿	class	설계도 (틀)
실제 레이어 1장	객체 (object)	설계도로 만든 실체
위치, 크기	필드 (field)	데이터
이동, 회전	메서드 (method)	동작

2. VS Code 환경 설정

2.1. 필요 도구 설치

1. .NET SDK 설치: <https://dotnet.microsoft.com/download>
2. Visual Studio Code 설치: <https://code.visualstudio.com>
3. VS Code 확장 설치: **C# Dev Kit** (Microsoft)

2.2. Console 프로젝트 생성

VS Code 터미널(Ctrl + `)에서 다음 명령을 실행한다.

프로젝트 생성 명령

```
dotnet new console -n Lecture4Class
cd Lecture4Class
code .
```

생성된 Program.cs에 다음 코드가 있는지 확인한다.

기본 Program.cs

```
1 // See https://aka.ms/new-console-template for more
  information
2 Console.WriteLine("Hello, World!");
```

참고

.NET 6 이후의 최상위 문(Top-level statements) 기능 덕분에 namespace, class Program, Main() 없이도 코드를 바로 실행할 수 있다. 이번 강의에서는 이 간결한 형태를 사용한다.

2.3. 빌드 및 실행

실행 명령

```
dotnet run
```

터미널에 Hello, World!가 출력되면 환경 설정이 완료된 것이다.

3. 클래스 기초 — 첫 번째 클래스 만들기

3.1. 클래스란?

클래스는 **관련된 데이터(필드)와 기능(메서드)을 하나로 묶은 설계도**이다. 이 설계도를 바탕으로 실제 메모리에 만들어진 것을 **객체(object)** 또는 **인스턴스(instance)**라 한다.

용어	비유
class (클래스)	붕어빵 틀
object (객체)	붕어빵 틀로 만든 실제 붕어빵
new 키워드	“틀에서 하나 찍어내기”

3.2. Step 1: 가장 간단한 클래스

색상(Color)을 표현하는 클래스를 만들어 보자. 게임 그래픽에서 색상은 R, G, B 세 개의 값으로 구성된다.

예제 — MyColor 클래스 (필드만)

```

1  // --- Class Definition ---
2  class MyColor
3  {
4      public int R;    // Red (0~255)
5      public int G;    // Green (0~255)
6      public int B;    // Blue (0~255)
7  }
8
9  // --- Usage (Program.cs top-level statements) ---
10 MyColor red = new MyColor();
11 red.R = 255;
12 red.G = 0;
13 red.B = 0;
14
15 Console.WriteLine("Red: R=" + red.R + " G=" + red.G + " B="
16                   + red.B);
17
18 MyColor sky = new MyColor();
19 sky.R = 135;
20 sky.G = 206;
21 sky.B = 235;
22
23 Console.WriteLine("Sky: R=" + sky.R + " G=" + sky.G + " B="
24                   + sky.B);

```

참고

public 키워드는 “외부에서 접근 가능”이라는 뜻이다. red.R처럼 점(.)으로 필드에 접근할 수 있는 이유가 바로 public 덕분이다.

실습 3-1: MyColor 클래스 실행

위 코드를 Program.cs에 입력하고 dotnet run으로 실행하라.

- 1) green이라는 이름의 MyColor 객체를 추가로 만들어 (0, 128, 0) 값을 설정하라.
- 2) 세 가지 색상의 정보를 모두 출력하라.

3.3. Step 2: 메서드 추가하기

클래스에 메서드(method)를 추가하면 객체가 “동작”을 가질 수 있다.

예제 — MyColor에 메서드 추가

```

1  class MyColor
2  {
3      public int R;
4      public int G;
5      public int B;
6
7      // Method that returns color info as a string
8      public string GetInfo()
9      {
10         return "RGB(" + R + ", " + G + ", " + B + ")";
11     }
12
13     // Method to calculate brightness (0.0 ~ 1.0)
14     public double GetBrightness()
15     {
16         return (R + G + B) / (255.0 * 3);
17     }
18 }
19
20 // --- Usage ---
21 MyColor sunset = new MyColor();
22 sunset.R = 255;
23 sunset.G = 99;
24 sunset.B = 71;
25
26 Console.WriteLine("Sunset: " + sunset.GetInfo());
27 Console.WriteLine("Brightness: " + Math.Round(sunset.
    GetBrightness(), 2));

```

실습 3-2: 메서드 추가 실습

MyColor 클래스에 다음 메서드를 추가하라.

- 1) bool IsGrayscale() — R, G, B 값이 모두 같으면 true, 아니면 false를 반환
- 2) string GetHex() — "#FF6347" 형태의 16진수 문자열을 반환
힌트: R.ToString("X2")는 정수를 2자리 16진수 문자열로 변환한다.

테스트: (128, 128, 128)은 Grayscale이고 Hex는 "#808080"이어야 한다.

4. 생성자와 프로퍼티

4.1. 생성자 (Constructor)

지금까지는 객체를 만든 뒤 필드를 하나씩 설정했다. **생성자**를 사용하면 객체 생성과 동시에 초기값을 설정할 수 있다.

예제 — 생성자가 있는 MyColor

```

1 class MyColor
2 {
3     public int R;
4     public int G;
5     public int B;
6
7     // Constructor: same name as class, no return type
8     public MyColor(int r, int g, int b)
9     {
10         R = r;
11         G = g;
12         B = b;
13     }
14
15     public string GetInfo()
16     {
17         return "RGB(" + R + ", " + G + ", " + B + ")";
18     }
19 }
20
21 // --- Usage: set values at creation ---
22 MyColor red    = new MyColor(255, 0, 0);
23 MyColor green  = new MyColor(0, 128, 0);
24 MyColor blue   = new MyColor(0, 0, 255);
25
26 Console.WriteLine(red.GetInfo());    // RGB(255, 0, 0)
27 Console.WriteLine(green.GetInfo()); // RGB(0, 128, 0)
28 Console.WriteLine(blue.GetInfo());   // RGB(0, 0, 255)

```

참고

생성자 덕분에 `new MyColor(255, 0, 0)` 한 줄로 객체 생성이 완료된다. 이전처럼 `red.R = 255; red.G = 0; red.B = 0;` 세 줄을 쓸 필요가 없다.

4.2. 프로퍼티 (Property)

`public` 필드는 외부에서 자유롭게 값을 바꿀 수 있어 위험할 수 있다. 예를 들어 `color.R = 999`처럼 유효하지 않은 값이 들어갈 수 있다.

프로퍼티는 필드에 대한 접근을 제어하는 “스마트 필드”이다.

예제 — 프로퍼티로 값 범위 보호

```

1 class MyColor
2 {
3     // private field: not directly accessible from outside
4     private int r;
5     private int g;

```

```

6     private int b;
7
8     // Property: access control via get/set
9     public int R
10    {
11        get { return r; }
12        set { r = Math.Clamp(value, 0, 255); }
13    }
14
15    public int G
16    {
17        get { return g; }
18        set { g = Math.Clamp(value, 0, 255); }
19    }
20
21    public int B
22    {
23        get { return b; }
24        set { b = Math.Clamp(value, 0, 255); }
25    }
26
27    public MyColor(int r, int g, int b)
28    {
29        R = r; // Set via property -> automatically
30              clamped to 0~255
31        G = g;
32        B = b;
33    }
34
35    public string GetInfo()
36    {
37        return "RGB(" + R + ", " + G + ", " + B + ")";
38    }
39
40    // --- Usage ---
41    MyColor c = new MyColor(300, -10, 128);
42    Console.WriteLine(c.GetInfo()); // RGB(255, 0, 128) ---
    auto-corrected!

```

자동 구현 프로퍼티

단순히 값을 저장/반환하기만 할 때는 간결한 문법을 사용할 수 있다.

```

1 public int R { get; set; } // compiler auto-generates a
    private field

```

범위 제한 등 특별한 로직이 필요하지 않다면 이 형태를 자주 사용한다.

실습 4-1: Pixel 클래스 만들기

화면의 한 점(픽셀)을 나타내는 Pixel 클래스를 작성하라.

요구사항:

- 프로퍼티: X (int), Y (int), Color (MyColor)
- 생성자: Pixel(int x, int y, MyColor color)
- 메서드: string GetInfo() — "Pixel(3, 7) = RGB(255, 0, 0)" 형태 반환
- 메서드: void MoveTo(int newX, int newY) — 위치 변경

```

1 class Pixel
2 {
3     public int X { get; set; }
4     public int Y { get; set; }
5     public MyColor Color { get; set; }
6
7     public Pixel(int x, int y, MyColor color)
8     {
9         X = x;
10        Y = y;
11        Color = color;
12    }
13
14    // TODO: Implement GetInfo() method
15    // TODO: Implement MoveTo() method
16 }
17
18 // Test
19 MyColor red = new MyColor(255, 0, 0);
20 Pixel p = new Pixel(3, 7, red);
21 Console.WriteLine(p.GetInfo()); // Pixel(3, 7) = RGB(255,
    0, 0)
22 p.MoveTo(10, 20);
23 Console.WriteLine(p.GetInfo()); // Pixel(10, 20) = RGB
    (255, 0, 0)

```

5. 클래스 활용 — 객체 배열과 상호작용

5.1. 객체 배열

여러 객체를 배열로 관리하면 대량의 데이터를 효율적으로 다룰 수 있다.

예제 — 팔레트(Palette): 색상 배열

```

1 class MyColor
2 {
3     public int R { get; set; }

```

```
4     public int G { get; set; }
5     public int B { get; set; }
6     public string Name { get; set; }
7
8     public MyColor(string name, int r, int g, int b)
9     {
10         Name = name; R = r; G = g; B = b;
11     }
12
13     public string GetInfo()
14     {
15         return Name + ": RGB(" + R + ", " + G + ", " + B +
16             ")";
17     }
18
19     public double GetBrightness()
20     {
21         return (R + G + B) / (255.0 * 3);
22     }
23
24     // --- Palette: array of MyColor objects ---
25     MyColor[] palette = new MyColor[]
26     {
27         new MyColor("Red",    255, 0,    0),
28         new MyColor("Orange", 255, 165, 0),
29         new MyColor("Yellow", 255, 255, 0),
30         new MyColor("Green",  0,    128, 0),
31         new MyColor("Blue",    0,    0,   255),
32         new MyColor("Indigo",  75,   0,   130),
33         new MyColor("Violet",  148, 0,   211),
34     };
35
36     Console.WriteLine("=== Rainbow Palette ===");
37     foreach (MyColor color in palette)
38     {
39         Console.WriteLine(color.GetInfo()
40             + " | Brightness: " + Math.Round(color.
41                 GetBrightness(), 2));
42     }
43
44     // Find the brightest color
45     MyColor brightest = palette[0];
46     for (int i = 1; i < palette.Length; i++)
47     {
48         if (palette[i].GetBrightness() > brightest.
49             GetBrightness())
50         {
51             brightest = palette[i];
52         }
53     }
```

```

51 }
52 Console.WriteLine("\nBrightest color: " + brightest.GetInfo());

```

실습 5-1: 팔레트 관리

위 팔레트 예제를 확장하라.

- 1) 가장 **어두운** 색을 찾아 출력하라.
- 2) 팔레트의 **평균 밝기**를 계산하여 출력하라.
- 3) 밝기 기준으로 “밝은 색(밝기 ≥ 0.5)”과 “어두운 색(밝기 < 0.5)”으로 분류하여 출력하라.

5.2. 클래스 안에서 다른 클래스 사용하기

하나의 클래스가 다른 클래스를 필드로 가질 수 있다. 이를 **컴포지션(Composition)**이라 한다.

예제 — GameCharacter 클래스

```

1  class Position
2  {
3      public int X { get; set; }
4      public int Y { get; set; }
5
6      public Position(int x, int y) { X = x; Y = y; }
7
8      public string GetInfo() { return "(" + X + ", " + Y + "
9          ); "; }
10 }
11
12 class GameCharacter
13 {
14     public string Name { get; set; }
15     public int Hp { get; set; }
16     public Position Pos { get; set; }    // Another class
17     as a field!
18
19     public GameCharacter(string name, int hp, int x, int y)
20     {
21         Name = name;
22         Hp = hp;
23         Pos = new Position(x, y);
24     }
25
26     public void Move(int dx, int dy)
27     {

```

```

26         Pos.X += dx;
27         Pos.Y += dy;
28         Console.WriteLine(Name + " moved -> " + Pos.GetInfo
           ());
29     }
30
31     public void TakeDamage(int damage)
32     {
33         Hp -= damage;
34         if (Hp < 0) Hp = 0;
35         Console.WriteLine(Name + " hit! HP: " + Hp);
36     }
37
38     public bool IsAlive()
39     {
40         return Hp > 0;
41     }
42
43     public string GetStatus()
44     {
45         string alive = IsAlive() ? "Alive" : "Dead";
46         return Name + " " + Pos.GetInfo()
47             + " HP:" + Hp + " [" + alive + "]";
48     }
49 }
50
51 // --- Usage ---
52 GameCharacter hero    = new GameCharacter("Hero", 100, 0, 0)
53     ;
54 GameCharacter goblin = new GameCharacter("Goblin", 30, 5,
55     3);
56
57 Console.WriteLine(hero.GetStatus());
58 Console.WriteLine(goblin.GetStatus());
59
60 hero.Move(3, 2);
61 goblin.TakeDamage(25);
62
63 Console.WriteLine("\n--- Status Update ---");
64 Console.WriteLine(hero.GetStatus());
65 Console.WriteLine(goblin.GetStatus());

```

실습 5-2: 전투 시스템

GameCharacter 클래스에 다음을 추가하라.

- 1) int Attack 프로퍼티 추가 (공격력)
- 2) void AttackTarget(GameCharacter target) 메서드 — 상대에게 자신의 공격력만큼 피해를 입힌다.

```

1 public void AttackTarget(GameCharacter target)
2 {
3     Console.WriteLine(Name + " -> " + target.Name + "
4         attacks!");
5     target.TakeDamage(Attack);
6 }

```

테스트: 용사와 고블린이 서로 번갈아 공격하여 한쪽이 쓰러질 때까지 반복하라.

```

1 GameCharacter hero    = new GameCharacter("Hero",    100, 0,
2     0);
3 GameCharacter goblin = new GameCharacter("Goblin",   30, 5,
4     3);
5 hero.Attack = 12;
6 goblin.Attack = 8;
7
8 int turn = 1;
9 while (hero.IsAlive() && goblin.IsAlive())
10 {
11     Console.WriteLine("--- Turn " + turn + " ---");
12     hero.AttackTarget(goblin);
13     if (goblin.IsAlive())
14         goblin.AttackTarget(hero);
15     turn++;
16 }
17
18 Console.WriteLine("\n=== Battle End ===");
19 Console.WriteLine(hero.GetStatus());
20 Console.WriteLine(goblin.GetStatus());

```

6. Console 시각화 — 텍스트로 그림 그리기

이제 Console 화면을 “캔버스” 처럼 활용하여 시각적인 출력을 만들어 보자. 이 기법은 이후 Game of Life 구현의 기반이 된다.

6.1. Console 색상 출력

예제 — Console 색상 블록 출력

```

1 // 16 colors available in Console
2 ConsoleColor[] colors = (ConsoleColor[])Enum.GetValues(
3     typeof(ConsoleColor));
4
5 foreach (ConsoleColor color in colors)
6 {
7     Console.BackgroundColor = color;
8     Console.Write("  "); // Color block (2 chars)
9 }

```

```

10 Console.ResetColor();
11 Console.WriteLine();
12 Console.WriteLine("16 default Console colors");

```

6.2. 커서 위치 제어

Console.SetCursorPosition(x, y)를 사용하면 화면의 원하는 위치에 출력할 수 있다.

예제 — 커서 위치로 도형 그리기

```

1 Console.Clear();
2
3 // Draw rectangle (10x5)
4 for (int y = 0; y < 5; y++)
5 {
6     Console.SetCursorPosition(5, y + 2);
7     for (int x = 0; x < 10; x++)
8     {
9         if (y == 0 || y == 4 || x == 0 || x == 9)
10        {
11            Console.BackgroundColor = ConsoleColor.Cyan;
12            Console.Write(" ");
13        }
14        else
15        {
16            Console.BackgroundColor = ConsoleColor.Black;
17            Console.Write(" ");
18        }
19    }
20 }
21
22 Console.ResetColor();
23 Console.SetCursorPosition(0, 10);
24 Console.WriteLine("Rectangle complete!");

```

6.3. Canvas 클래스 — 화면을 객체로 관리하기

Console 시각화를 클래스로 캡슐화하면 훨씬 깔끔하게 코드를 관리할 수 있다.

예제 — Canvas 클래스

```

1 class Canvas
2 {
3     private int width;
4     private int height;
5     private ConsoleColor[,] pixels; // 2D array
6
7     public Canvas(int width, int height)

```

```
8      {
9          this.width = width;
10         this.height = height;
11         pixels = new ConsoleColor[height, width];
12         Clear(ConsoleColor.Black);
13     }
14
15     // Fill entire screen with one color
16     public void Clear(ConsoleColor color)
17     {
18         for (int y = 0; y < height; y++)
19             for (int x = 0; x < width; x++)
20                 pixels[y, x] = color;
21     }
22
23     // Set a single pixel
24     public void SetPixel(int x, int y, ConsoleColor color)
25     {
26         if (x >= 0 && x < width && y >= 0 && y < height)
27             pixels[y, x] = color;
28     }
29
30     // Render to screen
31     public void Render()
32     {
33         Console.SetCursorPosition(0, 0);
34         for (int y = 0; y < height; y++)
35         {
36             for (int x = 0; x < width; x++)
37             {
38                 Console.BackgroundColor = pixels[y, x];
39                 Console.Write(" "); // 1 cell = 2 chars (
                                     closer to square)
40             }
41             Console.ResetColor();
42             Console.WriteLine();
43         }
44         Console.ResetColor();
45     }
46 }
47
48 // --- Usage: draw diagonal ---
49 Console.Clear();
50 Canvas canvas = new Canvas(20, 15);
51
52 for (int i = 0; i < 15; i++)
53 {
54     canvas.SetPixel(i, i, ConsoleColor.Yellow);
55 }
56
```

```

57 canvas.Render();
58 Console.SetCursorPosition(0, 17);
59 Console.WriteLine("Diagonal drawing complete!");

```

실습 6-1: Canvas로 그림 그리기

Canvas 클래스를 활용하여 다음을 그려 보라.

- 1) 빨간색 테두리가 있는 직사각형 (내부는 검정)
- 2) 초록색 대각선 두 개가 교차하는 X자
- 3) 자유 주제: 좋아하는 도형이나 간단한 캐릭터

힌트: SetPixel을 반복문 안에서 호출하라.

```

1  // Example: bordered rectangle
2  Canvas canvas = new Canvas(20, 15);
3
4  for (int x = 2; x < 18; x++)
5  {
6      canvas.SetPixel(x, 2, ConsoleColor.Red);    // top
7      canvas.SetPixel(x, 12, ConsoleColor.Red);    // bottom
8  }
9  for (int y = 2; y < 13; y++)
10 {
11     canvas.SetPixel(2, y, ConsoleColor.Red);    // left
12     canvas.SetPixel(17, y, ConsoleColor.Red);   // right
13 }
14
15 canvas.Render();

```

6.4. Canvas 클래스에 도형 메서드 추가하기

예제 — Canvas에 DrawRect, FillRect 추가

```

1  class Canvas
2  {
3      // ... (previous code omitted) ...
4
5      // Draw rectangle border
6      public void DrawRect(int x1, int y1, int x2, int y2,
7                          ConsoleColor color)
8      {
9          for (int x = x1; x <= x2; x++)
10         {
11             SetPixel(x, y1, color);    // top
12             SetPixel(x, y2, color);    // bottom
13         }

```



```

14         for (int y = y1; y <= y2; y++)
15         {
16             SetPixel(x1, y, color); // left
17             SetPixel(x2, y, color); // right
18         }
19     }
20
21     // Draw filled rectangle
22     public void FillRect(int x1, int y1, int x2, int y2,
23                         ConsoleColor color)
24     {
25         for (int y = y1; y <= y2; y++)
26             for (int x = x1; x <= x2; x++)
27                 SetPixel(x, y, color);
28     }
29 }
30
31 // --- Usage: simple scene ---
32 Console.Clear();
33 Canvas canvas = new Canvas(30, 18);
34
35 // sky
36 canvas.FillRect(0, 0, 29, 10, ConsoleColor.DarkCyan);
37 // ground
38 canvas.FillRect(0, 11, 29, 17, ConsoleColor.DarkGreen);
39 // house (wall)
40 canvas.FillRect(10, 6, 20, 14, ConsoleColor.DarkYellow);
41 // house (roof)
42 canvas.FillRect(8, 5, 22, 6, ConsoleColor.DarkRed);
43 // door
44 canvas.FillRect(14, 10, 16, 14, ConsoleColor.DarkMagenta);
45
46 canvas.Render();
47 Console.SetCursorPosition(0, 20);
48 Console.WriteLine("House drawing complete!");

```

실습 6-2: 나만의 장면 만들기

Canvas 클래스의 FillRect와 DrawRect를 활용하여 나만의 장면을 구성하라.
아이디어:

- 밤하늘에 별 (노란색 점들) + 달 (사각형 또는 점들)
- 바다 위에 떠 있는 배
- 로봇 얼굴 (네모난 눈, 입)

자유롭게 색상과 위치를 조합하여 표현해 보라.

7. 종합 프로젝트 — Conway's Game of Life

7.1. Game of Life란?

Conway's Game of Life는 수학자 John Conway가 1970년에 고안한 세포 자동자(Cellular Automaton)이다. 간단한 규칙만으로 복잡한 패턴이 생성되는 것이 특징이며, 게임 그래픽의 프로시저럴 생성(Procedural Generation)의 기초가 되는 개념이다.

7.1.1 규칙 (매우 간단!)

2차원 격자(Grid)의 각 칸(Cell)은 **살아있음(Alive)** 또는 **죽어있음(Dead)** 상태이다. 매 세대(Generation)마다 다음 규칙을 **모든 칸에 동시에** 적용한다.

현재 상태	이웃 수	다음 상태
살아있음 (Alive)	2 또는 3	생존 (유지)
살아있음 (Alive)	그 외	사망 (과밀 또는 고립)
죽어있음 (Dead)	정확히 3	탄생!
죽어있음 (Dead)	그 외	죽어있음 (유지)

여기서 “이웃”이란 상하좌우 + 대각선 방향의 8칸을 말한다.

7.2. Step 1: Cell 클래스

먼저 격자의 한 칸을 나타내는 Cell 클래스를 만든다.

Cell 클래스

```

1 class Cell
2 {
3     public bool IsAlive { get; set; }
4
5     public Cell(bool isAlive = false)
6     {
7         IsAlive = isAlive;
8     }
9 }
```

7.3. Step 2: Grid 클래스 — 격자 관리

Grid 클래스는 Cell 객체의 2차원 배열을 관리하고, Game of Life의 규칙을 적용하는 핵심 클래스이다.

Grid 클래스 — 기본 구조

```

1 class Grid
2 {
3     private int width;
4     private int height;
5     private Cell[,] cells;
6 }
```

```
7 public Grid(int width, int height)
8 {
9     this.width = width;
10    this.height = height;
11    cells = new Cell[height, width];
12
13    // Initialize all cells with Cell objects
14    for (int y = 0; y < height; y++)
15        for (int x = 0; x < width; x++)
16            cells[y, x] = new Cell(false);
17 }
18
19 // Set a specific cell to alive
20 public void SetAlive(int x, int y)
21 {
22     if (x >= 0 && x < width && y >= 0 && y < height)
23         cells[y, x].IsAlive = true;
24 }
25
26 // Count alive neighbors
27 private int CountNeighbors(int x, int y)
28 {
29     int count = 0;
30
31     // Search 8 directions
32     for (int dy = -1; dy <= 1; dy++)
33     {
34         for (int dx = -1; dx <= 1; dx++)
35         {
36             if (dx == 0 && dy == 0) continue; //
37                                     Exclude self
38
39             int nx = x + dx;
40             int ny = y + dy;
41
42             // Check grid bounds
43             if (nx >= 0 && nx < width && ny >= 0 && ny
44                 < height)
45             {
46                 if (cells[ny, nx].IsAlive)
47                     count++;
48             }
49         }
50     }
51     return count;
52 }
53
54 // Advance to next generation
55 public void NextGeneration()
56 {
```

```
55     Cell[,] next = new Cell[height, width];
56
57     for (int y = 0; y < height; y++)
58     {
59         for (int x = 0; x < width; x++)
60         {
61             int neighbors = CountNeighbors(x, y);
62             bool alive = cells[y, x].IsAlive;
63
64             // Apply Game of Life rules
65             if (alive && (neighbors == 2 || neighbors
66                 == 3))
67                 next[y, x] = new Cell(true);    // Survive
68             else if (!alive && neighbors == 3)
69                 next[y, x] = new Cell(true);    // Birth
70             else
71                 next[y, x] = new Cell(false);   // Death or remain dead
72         }
73     }
74
75     cells = next; // Replace with new generation
76
77     // Render grid to Console
78     public void Render()
79     {
80         Console.SetCursorPosition(0, 0);
81         for (int y = 0; y < height; y++)
82         {
83             for (int x = 0; x < width; x++)
84             {
85                 if (cells[y, x].IsAlive)
86                 {
87                     Console.BackgroundColor = ConsoleColor.
88                         Green;
89                     Console.Write(" ");
90                 }
91                 else
92                 {
93                     Console.BackgroundColor = ConsoleColor.
94                         Black;
95                     Console.Write(" ");
96                 }
97             }
98             Console.ResetColor();
99             Console.WriteLine();
100         }
101         Console.ResetColor();
102     }
```

```

100     }
101 }

```

7.4. Step 3: 초기 패턴 설정

Game of Life의 재미는 초기 패턴에 따라 전혀 다른 결과가 나온다는 것이다.

유명한 초기 패턴들

```

1  class Grid
2  {
3      // ... (previous code) ...
4
5      // Glider: pattern that moves diagonally
6      public void PlaceGlider(int startX, int startY)
7      {
8          SetAlive(startX + 1, startY);
9          SetAlive(startX + 2, startY + 1);
10         SetAlive(startX,      startY + 2);
11         SetAlive(startX + 1, startY + 2);
12         SetAlive(startX + 2, startY + 2);
13     }
14
15     // Blinker: oscillating pattern (period 2)
16     public void PlaceBlinker(int startX, int startY)
17     {
18         SetAlive(startX,      startY);
19         SetAlive(startX + 1, startY);
20         SetAlive(startX + 2, startY);
21     }
22
23     // Block: still life pattern (unchanged)
24     public void PlaceBlock(int startX, int startY)
25     {
26         SetAlive(startX,      startY);
27         SetAlive(startX + 1, startY);
28         SetAlive(startX,      startY + 1);
29         SetAlive(startX + 1, startY + 1);
30     }
31
32     // Random initialization
33     public void Randomize(double density = 0.3)
34     {
35         Random rand = new Random();
36         for (int y = 0; y < height; y++)
37             for (int x = 0; x < width; x++)
38                 cells[y, x].IsAlive = rand.NextDouble() <
39                     density;

```

```
40 }
```

7.5. Step 4: 실행 — 모든 것을 합치기

완성된 Program.cs

```
1  // =====
2  // Conway's Game of Life - C# Console Version
3  // =====
4
5  Console.CursorVisible = false;
6  Console.Clear();
7
8  int gridWidth  = 40;
9  int gridHeight = 20;
10
11 Grid grid = new Grid(gridWidth, gridHeight);
12
13 // Place initial patterns
14 grid.PlaceGlider(2, 2);
15 grid.PlaceGlider(10, 5);
16 grid.PlaceBlinker(20, 10);
17 grid.PlaceBlock(30, 3);
18
19 int generation = 0;
20
21 while (true)
22 {
23     grid.Render();
24
25     // Display info at bottom
26     Console.SetCursorPosition(0, gridHeight + 1);
27     Console.ForegroundColor = ConsoleColor.White;
28     Console.Write("Generation: " + generation
29         + " | Q: Quit R: Randomize Space: Pause");
30     Console.ResetColor();
31
32     // Check key input (non-blocking)
33     if (Console.KeyAvailable)
34     {
35         ConsoleKey key = Console.ReadKey(true).Key;
36         if (key == ConsoleKey.Q) break;
37         if (key == ConsoleKey.R)
38         {
39             grid = new Grid(gridWidth, gridHeight);
40             grid.Randomize(0.3);
41             generation = 0;
42         }
43     }
44 }
```

```

43         if (key == ConsoleKey.Spacebar)
44         {
45             Console.ReadKey(true); // Wait until any key is
                                   // pressed
46         }
47     }
48
49     grid.NextGeneration();
50     generation++;
51     Thread.Sleep(150); // Wait 150ms (speed control)
52 }
53
54 Console.CursorVisible = true;
55 Console.Clear();
56 Console.WriteLine("Game of Life ended. Total " + generation
                    + " generations.");

```

Thread.Sleep 사용

Thread.Sleep(150)은 프로그램을 150밀리초 동안 멈추게 한다. 이 값을 줄이면 빠르게, 늘리면 느리게 세대가 진행된다. using System.Threading; 없이도 최상위 문에서 바로 사용 가능하다.

실습 7-1: Game of Life 실행 및 관찰

위 코드를 조합하여 프로젝트를 완성하고 실행하라.

- 1) 글라이더가 대각선으로 이동하는 것을 관찰하라.
- 2) 블링커가 가로/세로로 깜박이는 것을 확인하라.
- 3) 블록이 변하지 않는 것을 확인하라.
- 4) R 키를 눌러 랜덤 초기화 후 패턴 변화를 관찰하라.

실습 7-2: 새로운 패턴 추가

Grid 클래스에 다음 패턴을 추가하는 메서드를 작성하라.

1. Toad (두꺼비) — 주기 2 진동 패턴:

```

. # # #
# # # .

```

```

1 public void PlaceToad(int startX, int startY)
2 {
3     // TODO: Call SetAlive according to the pattern above
4 }

```

2. Lightweight Spaceship (LWSS) — 이동하는 패턴:

```

. # # # #
# . . . #
. . . . #
# . . # .

```

```

1 public void PlaceLWSS(int startX, int startY)
2 {
3     // TODO: Call SetAlive according to the pattern above
4 }

```

도전 과제 1: 세대 통계 표시

Grid 클래스에 다음 메서드를 추가하라.

- `int CountAlive()` — 살아있는 셀의 총 수 반환

매 세대마다 하단에 살아있는 셀 수를 표시하고, 연속 3세대 동안 살아있는 셀 수가 변하지 않으면 “안정 상태 도달!”을 출력하라.

도전 과제 2: 색상으로 나이 표현하기

Cell 클래스에 `int Age` 프로퍼티를 추가하라. 살아있는 동안 매 세대마다 Age가 1씩 증가한다.

Render 시 Age에 따라 다른 색상을 사용하라:

Age	색상
1	<code>ConsoleColor.DarkGreen</code>
2-4	<code>ConsoleColor.Green</code>
5-9	<code>ConsoleColor.Yellow</code>
10 이상	<code>ConsoleColor.Red</code>

힌트: `NextGeneration()`에서 생존 시 이전 Age를 이어받고 +1 하라.

도전 과제 3: 마우스 클릭으로 셀 토글 (심화)

프로그램 시작 시 “편집 모드”를 추가하라.

- 화살표 키로 커서 이동
- Enter 키로 현재 위치의 셀 생존/사망 토글
- S 키로 시뮬레이션 시작

힌트: `Console.ReadKey()`로 화살표 키 입력을 받고, 커서 위치를 변수로 관리하라.

8. 핵심 개념 정리

개념	핵심 내용
클래스 (class)	데이터(필드) + 기능(메서드)을 묶은 설계도
객체 (object)	new 키워드로 클래스에서 만든 실체
필드 (field)	객체가 가진 데이터 (public int R;)
메서드 (method)	객체가 할 수 있는 동작 (public string GetInfo())
생성자 (constructor)	객체 생성 시 초기값 설정 (public MyColor(int r, ...))
프로퍼티 (property)	필드 접근을 제어하는 스마트 필드 (get; set;)
컴포지션	클래스 안에 다른 클래스를 필드로 포함
private / public	접근 제한: 내부 전용 / 외부 공개

이번 강의의 클래스들	역할
MyColor	색상 데이터 캡슐화 (필드, 메서드, 생성자, 프로퍼티 학습)
Pixel	위치 + 색상 조합 (컴포지션 학습)
GameCharacter	객체 간 상호작용 (메서드로 다른 객체 조작)
Canvas	2차원 배열 + Console 시각화 (캡슐화 학습)
Cell + Grid	Game of Life (클래스 설계 종합)

다음 강의 예고

Lecture 5에서는 상속(Inheritance)과 다형성(Polymorphism)을 학습한다.

- 클래스 상속과 override
- 추상 클래스와 인터페이스
- Unity 컴포넌트 시스템과의 연결